

A Human Comparison of GPT-4.1, LLaVA-7B, and Qwen2.5-VL

May 8, 2026

1 What This Report Is Actually Comparing

This report compares the three strongest models in the current repository benchmark sweep: `gpt-4.1`, `llava-7b`, and `qwen25-v1`. The quantitative results come directly from the regenerated comparison outputs in `comparison/output/`. In the local codebase, `qwen25-v1` maps to `Qwen/Qwen2.5-VL-3B-Instruct`, while `llava-7b` maps to `llava-hf/llava-1.5-7b-hf`. The OpenAI wrapper in `models/gpt4.py` currently defaults to `gpt-4o`, but the saved benchmark files and the regenerated comparison report include a separate `gpt-4.1` result set, so this document follows the comparison outputs rather than the wrapper default.

The benchmark suite covers 21 saved tasks across four families: captioning, question answering, labeling, and detection. The comparison report was rebuilt locally on May 8, 2026 using:

```
$env:PYTHONPATH='.'; python comparison\build_report.py
```

2 Short Version

If you only want the conclusion, it is this:

- `gpt-4.1` is the best all-around model in this run if you care most about average score and labeling-heavy tasks.
- `qwen25-v1` is the most consistently competitive model, wins the most individual benchmarks, and is especially strong on captioning.
- `llava-7b` is the best open-weight baseline here in the sense that it stays surprisingly close to the leaders on many tasks, but it is not quite as broad or as robust.

That is the simple story. The more interesting story is *why* they separate the way they do.

3 Headline Numbers

Table 1: Overall comparison numbers from `comparison/output/model_summary.csv`.

Model	Mean score	Normalized	Mean rank	1st places
<code>gpt-4.1</code>	0.518	0.758	2.10	10
<code>qwen25-v1</code>	0.478	0.727	1.95	11
<code>llava-7b</code>	0.467	0.661	2.14	7

These three models are closer than a casual glance suggests. `gpt-4.1` has the best average score, but `qwen25-v1` has the best average rank and the most first-place finishes. That means

gpt-4.1 is slightly better *on average*, while **qwen25-v1** is more often at or near the top of a given benchmark. **llava-7b** is a small step behind both, but not by enough to treat it as a different class of model.

Table 2: Family-level performance from `comparison/output/family_summary.csv`.

Model	Family	Mean score	Mean rank	Mean normalized
qwen25-v1	captioning	0.325	1.50	0.977
llava-7b	captioning	0.325	1.75	0.936
gpt-4.1	captioning	0.260	3.50	0.715
qwen25-v1	qa	0.533	2.00	0.726
llava-7b	qa	0.367	3.33	0.433
gpt-4.1	qa	0.400	3.67	0.333
gpt-4.1	labeling	0.764	1.36	0.877
llava-7b	labeling	0.673	1.82	0.712
qwen25-v1	labeling	0.618	2.27	0.627
gpt-4.1	detection	0.080	1.33	0.806
qwen25-v1	detection	0.111	1.33	0.765
llava-7b	detection	0.000	2.67	0.333

4 How The Three Models Work Internally

4.1 GPT-4.1

OpenAI does not publish a full layer-by-layer architecture for **gpt-4.1**, so the diagram below is the public-facing view rather than a reverse-engineered claim. What is public is that **gpt-4.1** accepts image input, uses a very long context window, and is optimized for strong instruction following and tool use.¹ The local wrapper in `models/gpt4.py` sends an image and prompt to the Responses API as a multimodal user message.

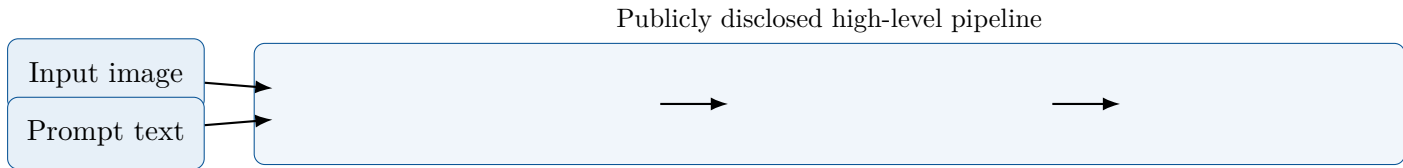


Figure 1: High-level view of the **gpt-4.1** inference path used in this repo.

4.2 LLaVA-7B

LLaVA-1.5-7B is much more transparent. Its standard recipe is a CLIP-style vision encoder, a learned projector that maps image features into the language model token space, and a Vicuna/LLaMA-family decoder-style transformer fine-tuned on visual instruction data.² In this repo, the wrapper loads `LlavaForConditionalGeneration` plus an `AutoProcessor`, prepares a chat-style prompt, and generates text autoregressively.

¹<https://developers.openai.com/api/docs/models/gpt-4.1>

²<https://huggingface.co/llava-hf/llava-1.5-7b-hf>

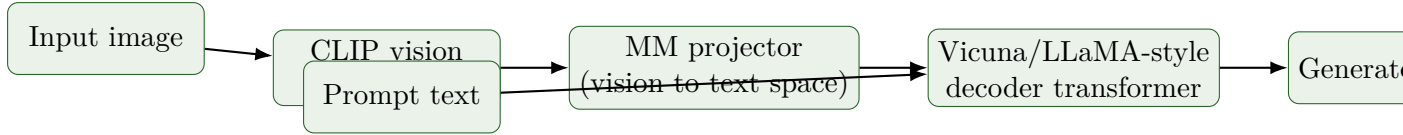


Figure 2: Typical LLaVA-1.5-7B architecture, which matches the repo’s `llava-hf/llava-1.5-7b-hf` integration.

4.3 Qwen2.5-VL

Qwen2.5-VL is also reasonably well documented. The public technical description emphasizes a stronger vision stack with dynamic resolution handling, window attention in the ViT, and an updated MRoPE positional mechanism designed to preserve spatial and temporal structure better than earlier variants.³ In this repo, the model is loaded through `Qwen2_5_VLForConditionalGeneration` and a multimodal chat template.

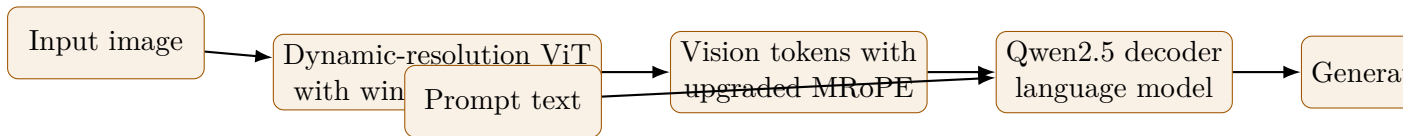


Figure 3: High-level Qwen2.5-VL pipeline reflected in the repo’s `qwen25-vl` wrapper.

5 Why The Scores Separate

5.1 Why GPT-4.1 finishes first overall

The easiest way to say it is that `gpt-4.1` looks like the most *general-purpose polished* system of the three. It does not dominate every family, but it does the fewest obviously weak things. In the saved comparison outputs, it is strongest on labeling and edges out the field overall on average score.

That pattern makes sense. Labeling benchmarks reward robust visual recognition, careful prompt following, and stable output formatting. Proprietary frontier systems usually have an advantage here because they are trained at larger scale, tuned heavily for instruction reliability, and optimized around product use cases where small formatting failures matter.

There is also a softer point: `gpt-4.1` appears to degrade gracefully. It may not win every benchmark outright, but it avoids the sharp collapses that drag down open-weight models when the prompt format, class granularity, or image distribution shifts.

5.2 Why Qwen2.5-VL wins more individual benchmarks

`qwen25-vl` has the best mean rank and the most first-place finishes. That usually means the model has a high ceiling and a strong multimodal alignment stack, even if it is a little less even across the whole suite than `gpt-4.1`.

The family breakdown supports that reading. Qwen is best on captioning and remains strong everywhere else. Captioning tends to reward rich image grounding, fluent generation, and the ability to turn visual details into natural text without sounding formulaic. That is exactly the sort of behavior that a stronger native multimodal design can improve.

Its public design choices also line up with the observed results. A better vision encoder path, dynamic resolution handling, and stronger positional treatment should help when the image itself carries a lot of the useful information and the model needs to preserve more of that structure before generation.

³https://huggingface.co/docs/transformers/en/model_doc/qwen2_5_vl

5.3 Why LLaVA-7B stays close but does not quite lead

`llava-7b` is the most interpretable model of the three in the sense that its strengths and weaknesses look like what you would expect from the architecture. It is built from a good vision encoder plus a good language model joined by a projector. That design works, and it works surprisingly well, but it still has a bottleneck at the interface between vision features and the language backbone.

In plain English, LLaVA often feels like a strong text model that has learned to look, rather than a model built around vision from the start. That is not meant as a criticism. It is exactly why it can be so competitive for its size. But it also helps explain why it lands a bit behind `gpt-4.1` and `qwen25-v1` when the task needs very precise visual grounding or more robust multimodal reasoning across varied benchmark styles.

5.4 Why the ranking changes by task family

The ranking changes because the benchmark families ask for different kinds of competence:

- Captioning rewards expressive grounded generation. Qwen does best here, with LLaVA close behind.
- Labeling rewards disciplined classification behavior and clean mapping from image to exact label inventory. GPT-4.1 is clearly best here.
- Detection is unstable for all three because this benchmark is both format-sensitive and spatially demanding. GPT-4.1 and Qwen are competitive; LLaVA collapses harder.
- QA in this suite is noisy enough that the differences should not be over-read, but GPT-4.1 is not the leader there and Qwen remains stronger overall than the raw QA row alone would suggest.

6 A Note About Speed

The telemetry should be read carefully. In the comparison output, `llava-7b` and `gpt-4.1` are the fastest of the three, while `qwen25-v1` is still fairly quick:

Table 3: Telemetry from `comparison/output/telemetry_model_summary.csv`.

Model	Mean wall-clock (s)	Mean generation (s)	Peak CPU RAM (GiB)
<code>llava-7b</code>	1.19	1.11	4.34
<code>gpt-4.1</code>	1.42	1.35	4.54
<code>qwen25-v1</code>	2.70	2.58	4.03

That said, this is not a perfectly apples-to-apples systems benchmark. `gpt-4.1` is called through a remote API, so the local machine is not doing the same kind of work it is doing for the two open-weight models. The speed numbers are still useful operationally, but they are not pure model-efficiency measurements.

7 Pairwise Reading

The pairwise win table says something subtle and important:

- `qwen25-v1` beats `llava-7b` on 11 of 21 shared benchmarks.

- `qwen25-v1` beats `gpt-4.1` on 8 of 21, while `gpt-4.1` beats Qwen on 7 of 21, with 6 ties.
- `llava-7b` beats `gpt-4.1` on 7 of 21, with 8 ties.

So this is not a clean ladder where one model simply dominates the next. It is more like this:

- `gpt-4.1` is the safest overall pick.
- `qwen25-v1` is the sharpest multimodal specialist among the three.
- `llava-7b` is credible enough that any serious evaluation should keep it in the room.

8 Practical Takeaways

If I had to summarize the three models in ordinary language:

- `gpt-4.1` is the one I would trust most when I want the fewest surprises.
- `qwen25-v1` is the one I would reach for when image grounding and natural visual description matter a lot.
- `llava-7b` is the one that proves how far a simpler open architecture can go before the more integrated multimodal systems start to pull away.

9 Comparison Graphs

The figures below are the graphs generated by the comparison pipeline and included here directly from `comparison/output/`. They are presented with only light commentary because the plots mostly speak for themselves.

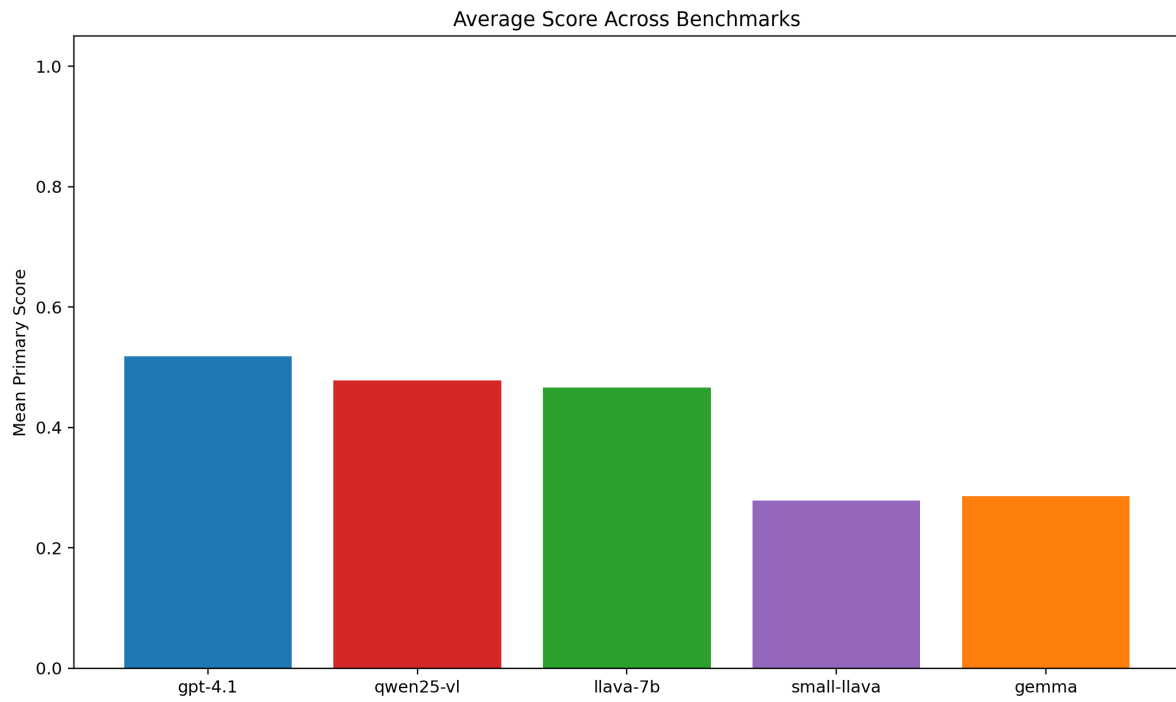


Figure 4: Mean raw scores by model.

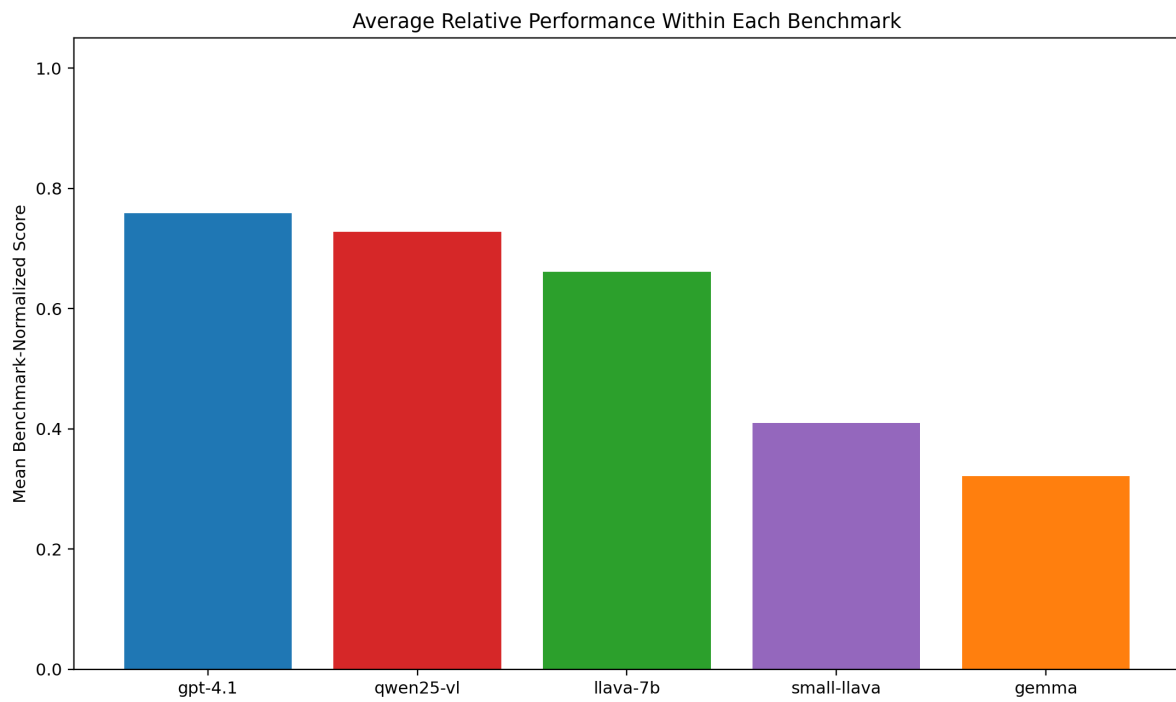


Figure 5: Mean benchmark-normalized scores by model.

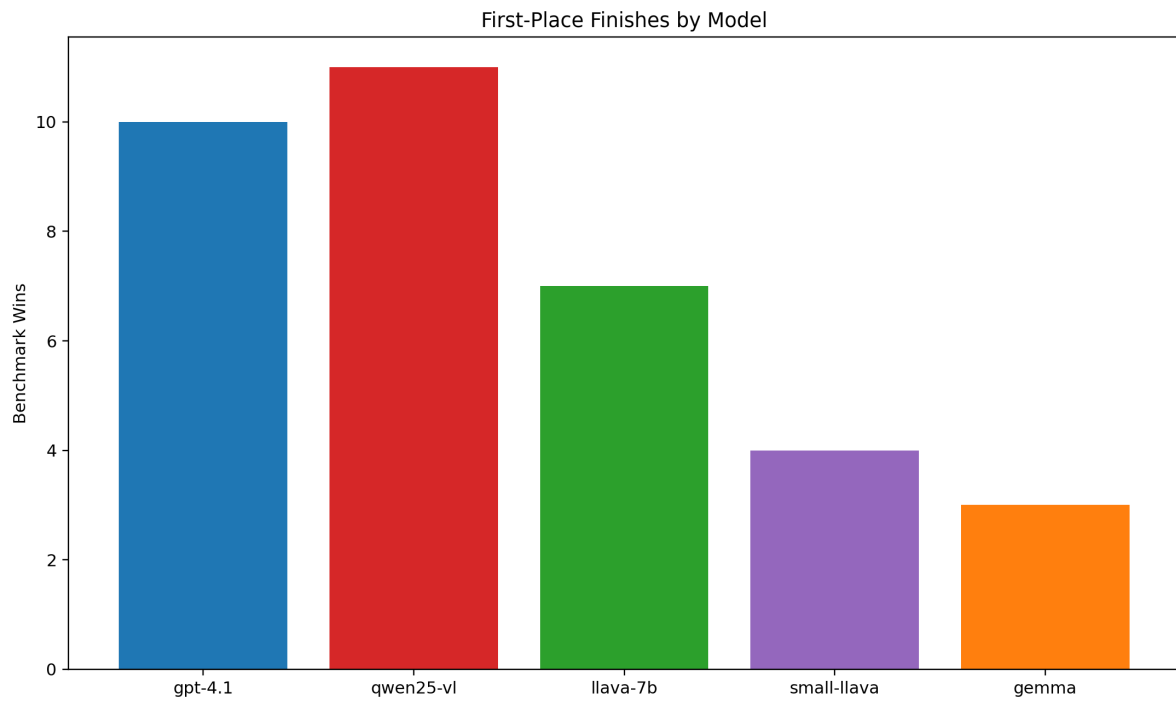


Figure 6: First-place finishes by model.

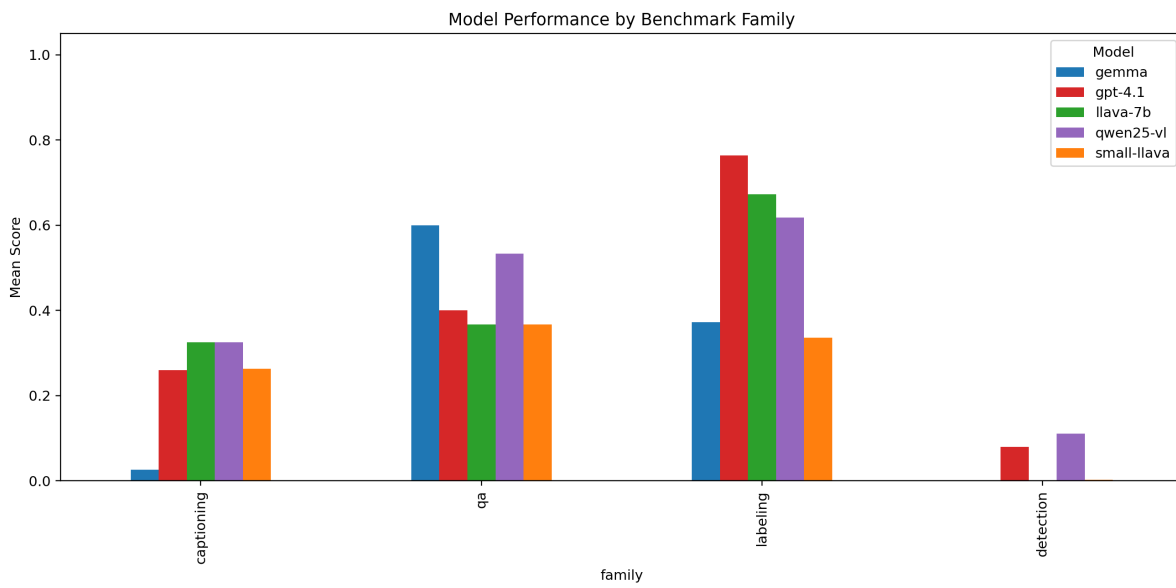


Figure 7: Grouped family comparison.

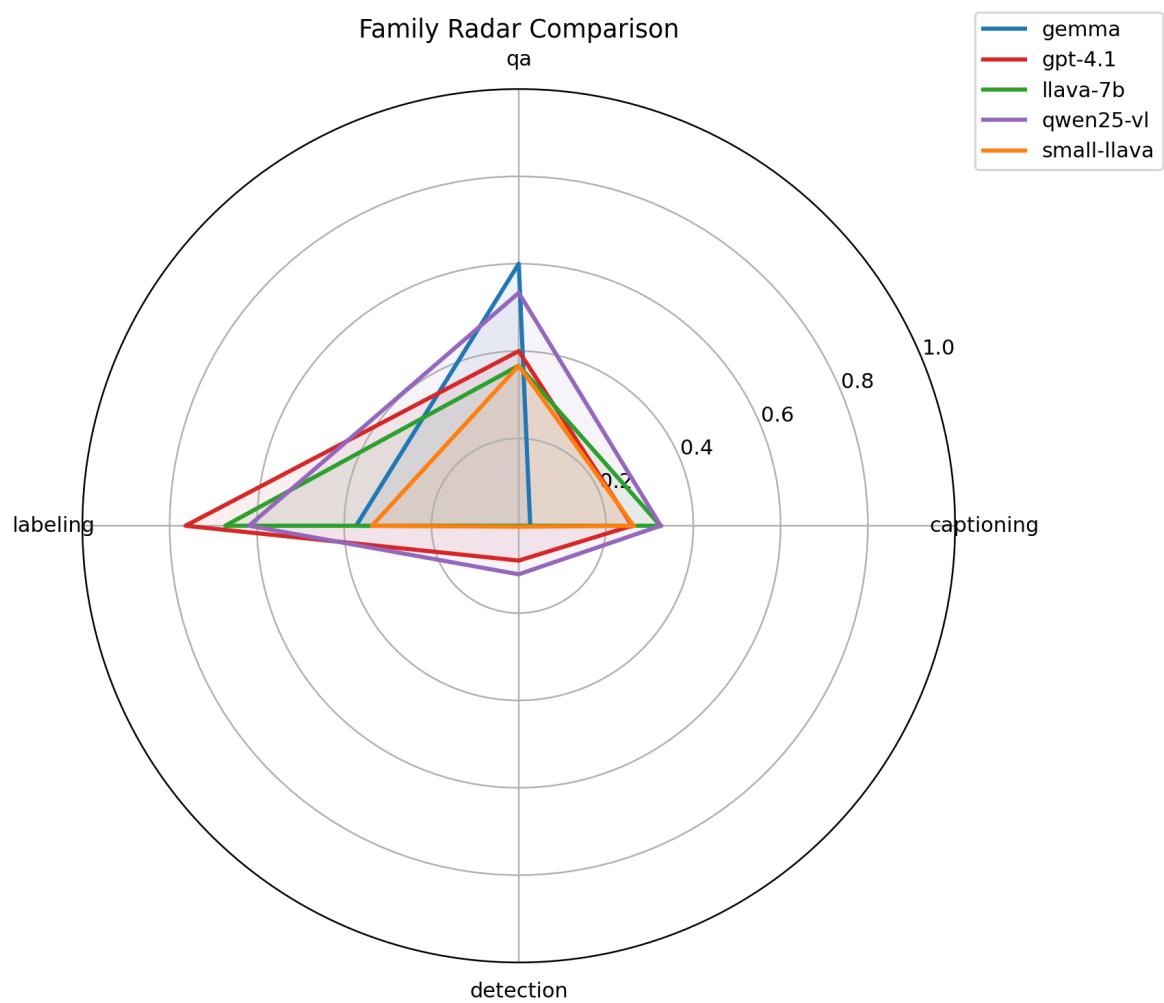


Figure 8: Family radar comparison.

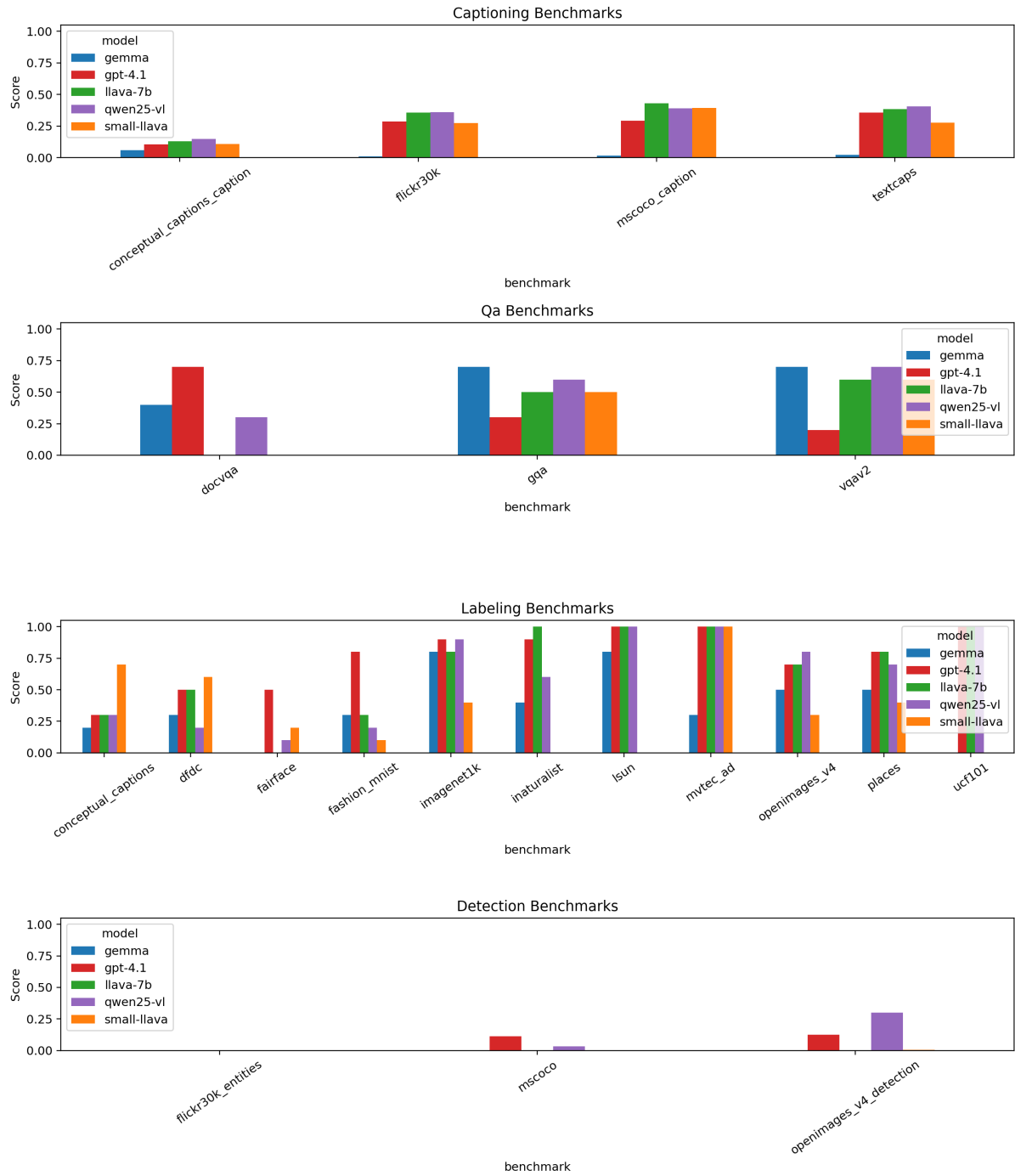


Figure 9: Family small multiples.

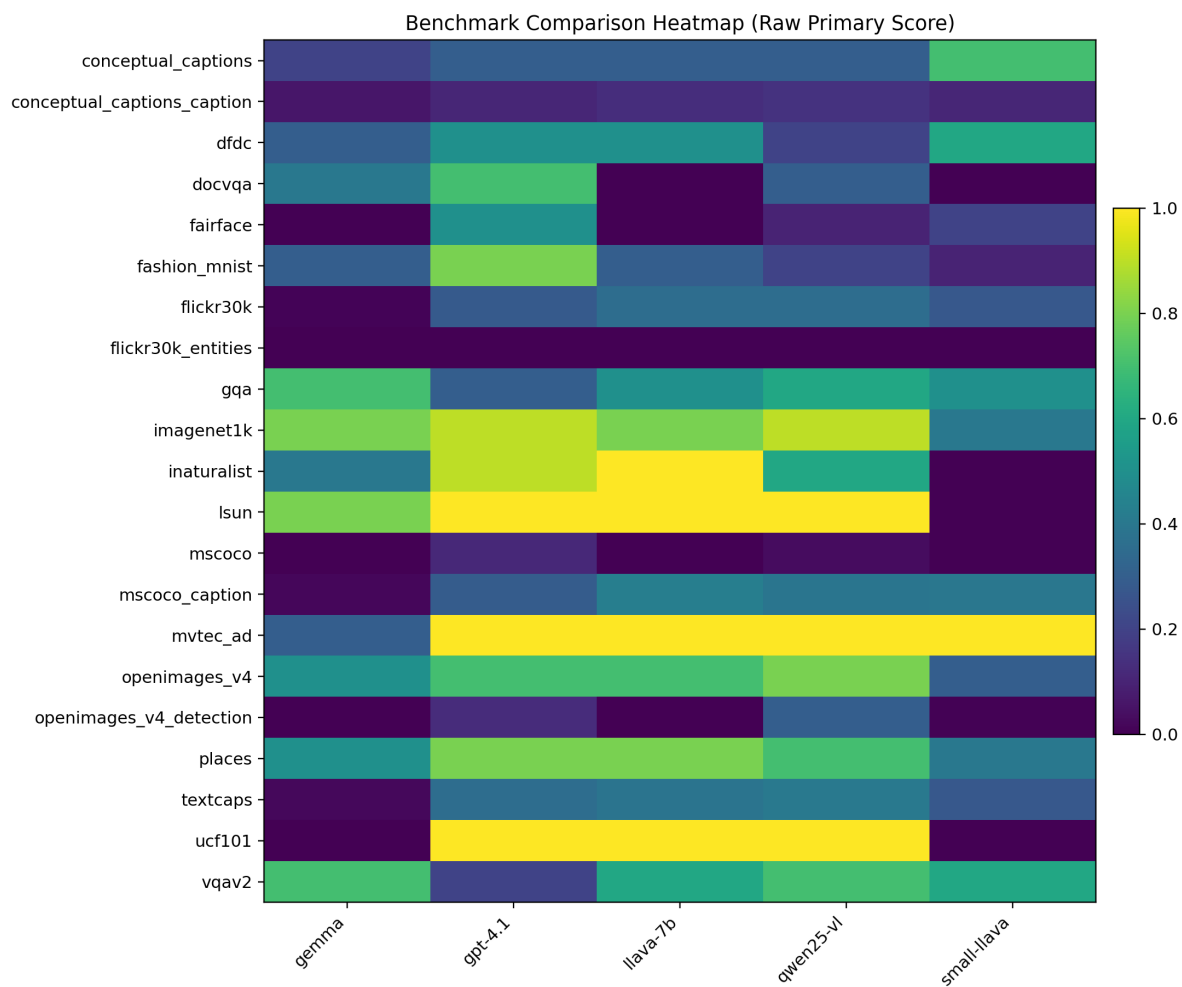


Figure 10: Raw benchmark score heatmap.

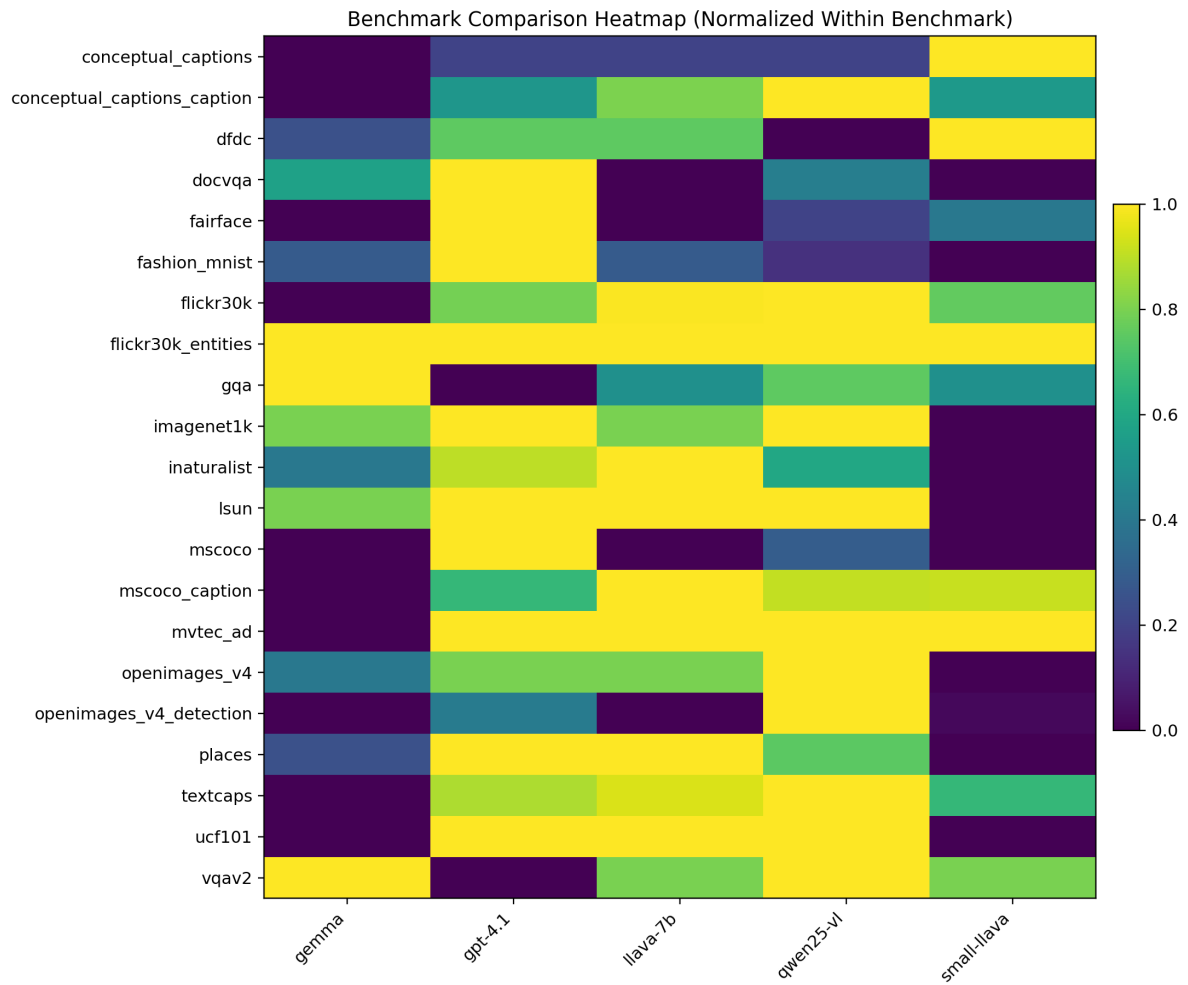


Figure 11: Normalized benchmark score heatmap.

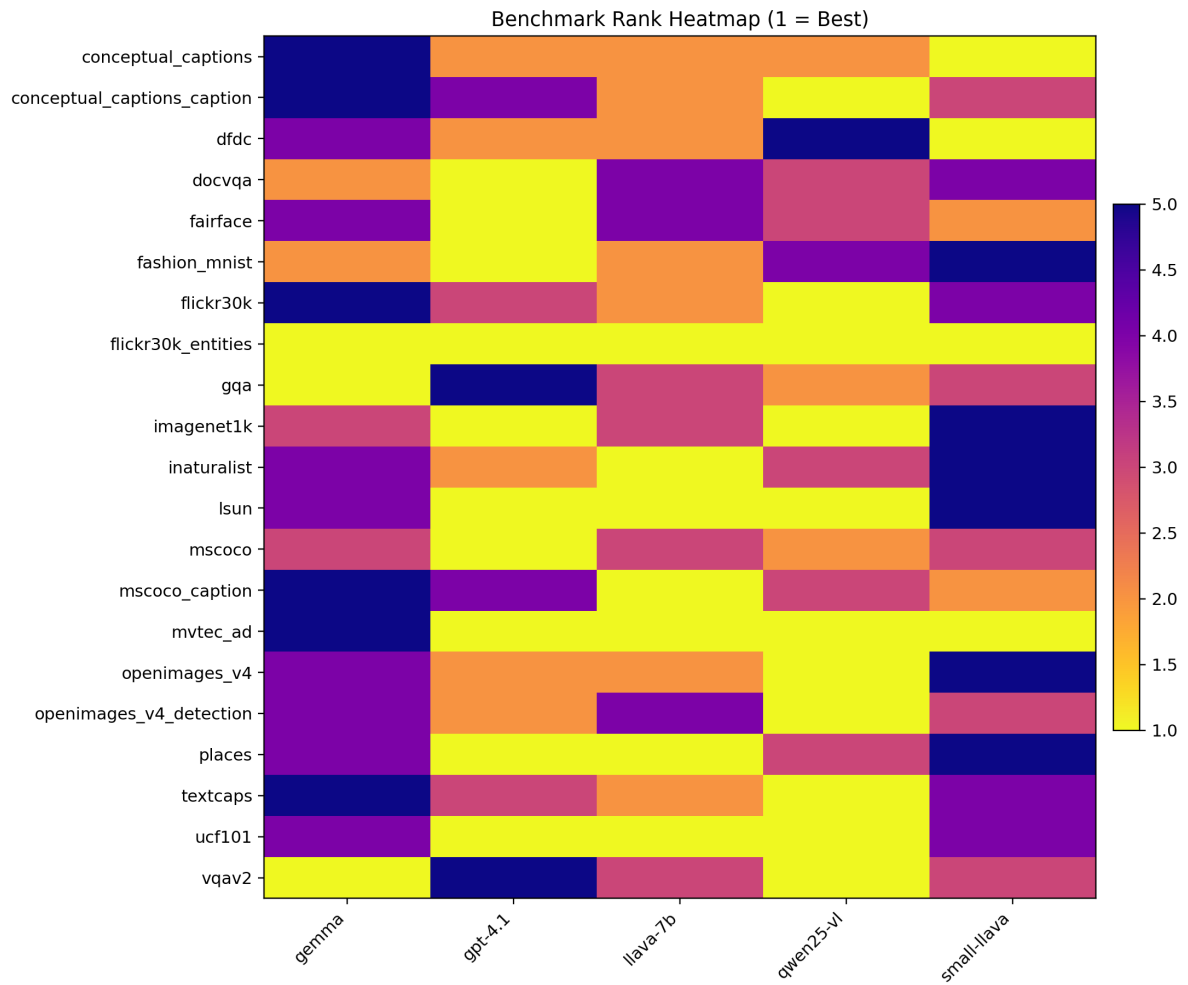


Figure 12: Benchmark rank heatmap.

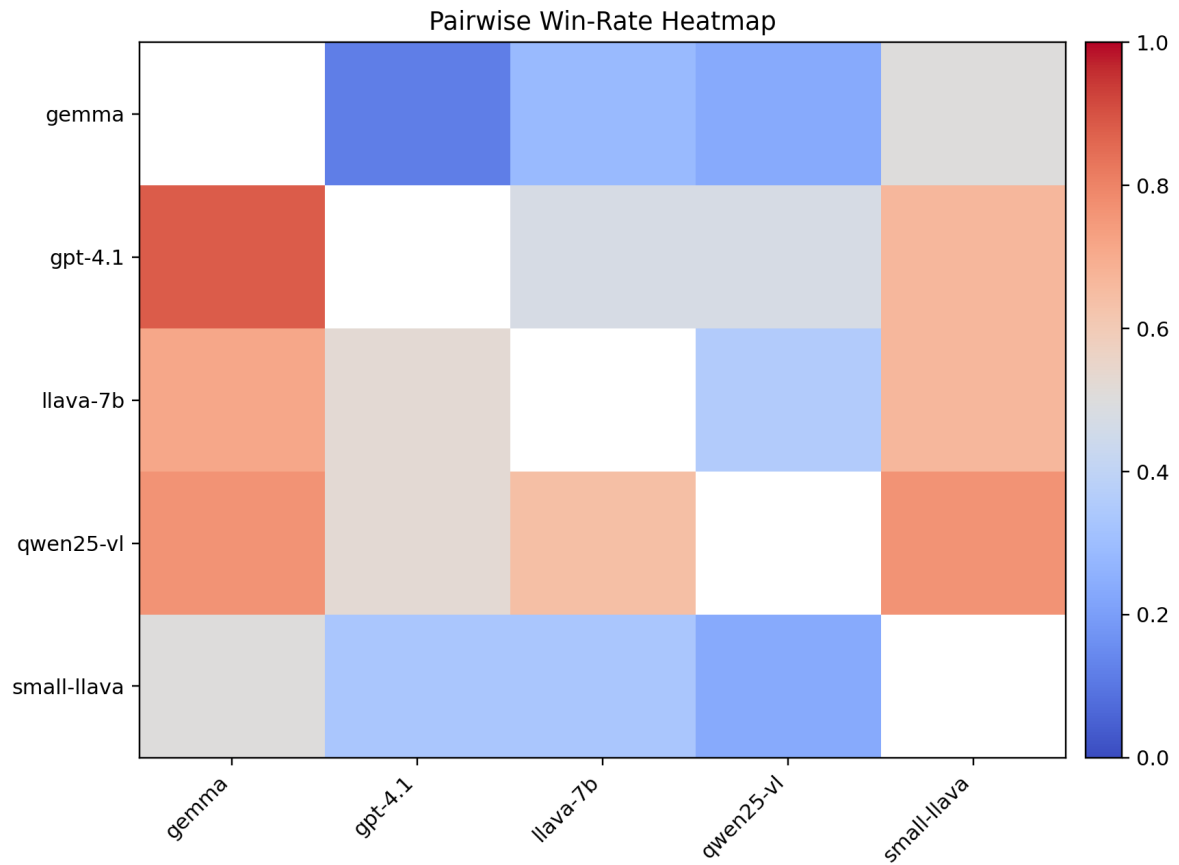


Figure 13: Pairwise win-rate heatmap.

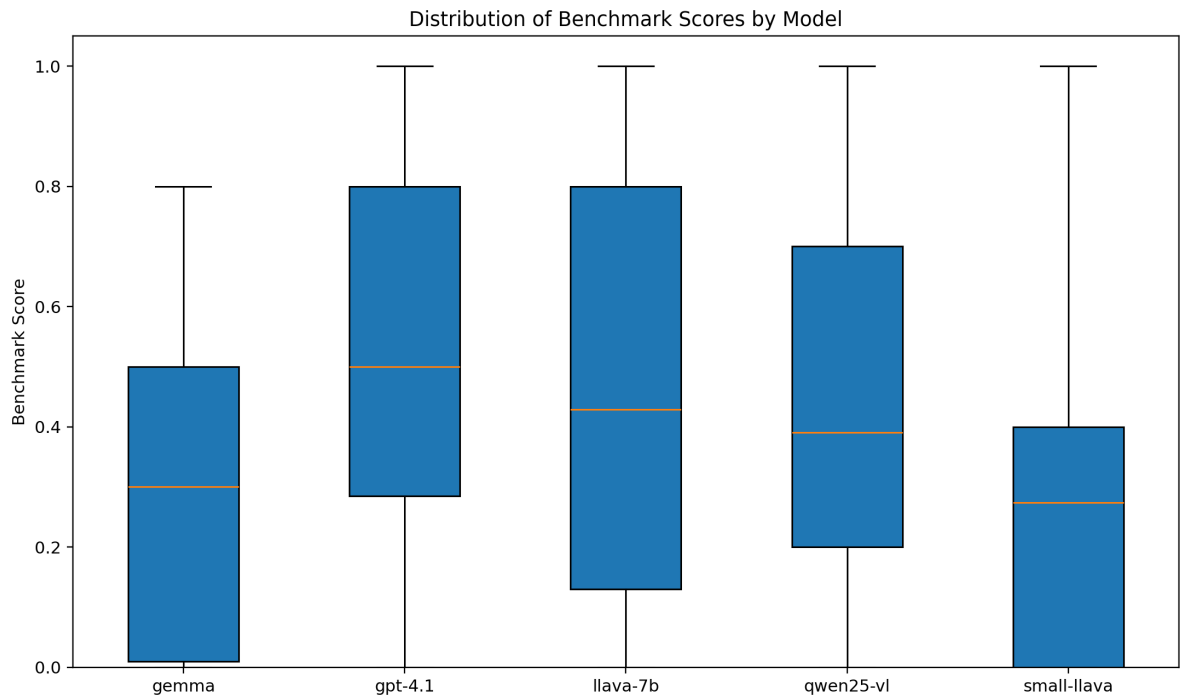


Figure 14: Score distribution boxplot.

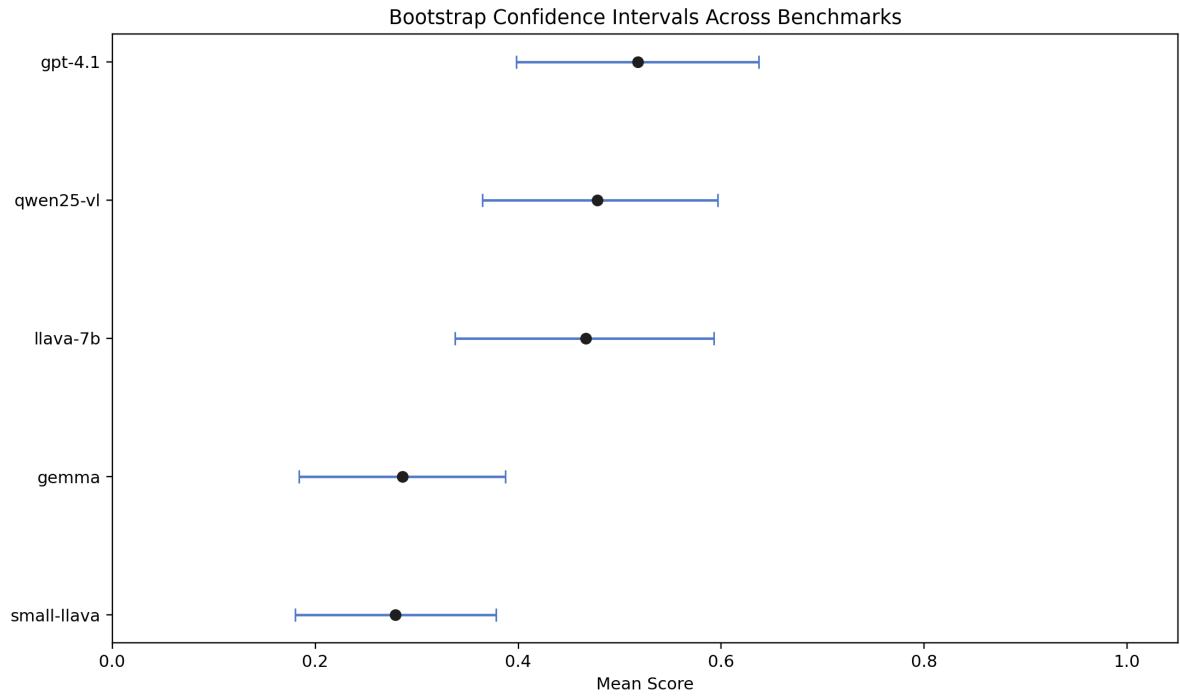


Figure 15: Bootstrap confidence intervals over benchmarks.

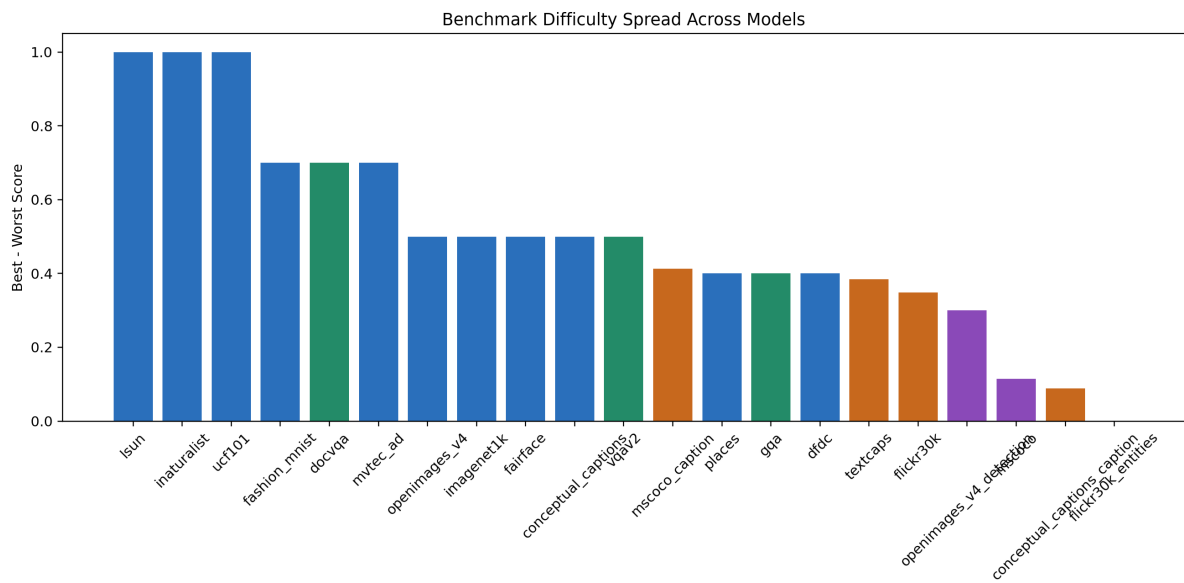


Figure 16: Benchmark spread across models.

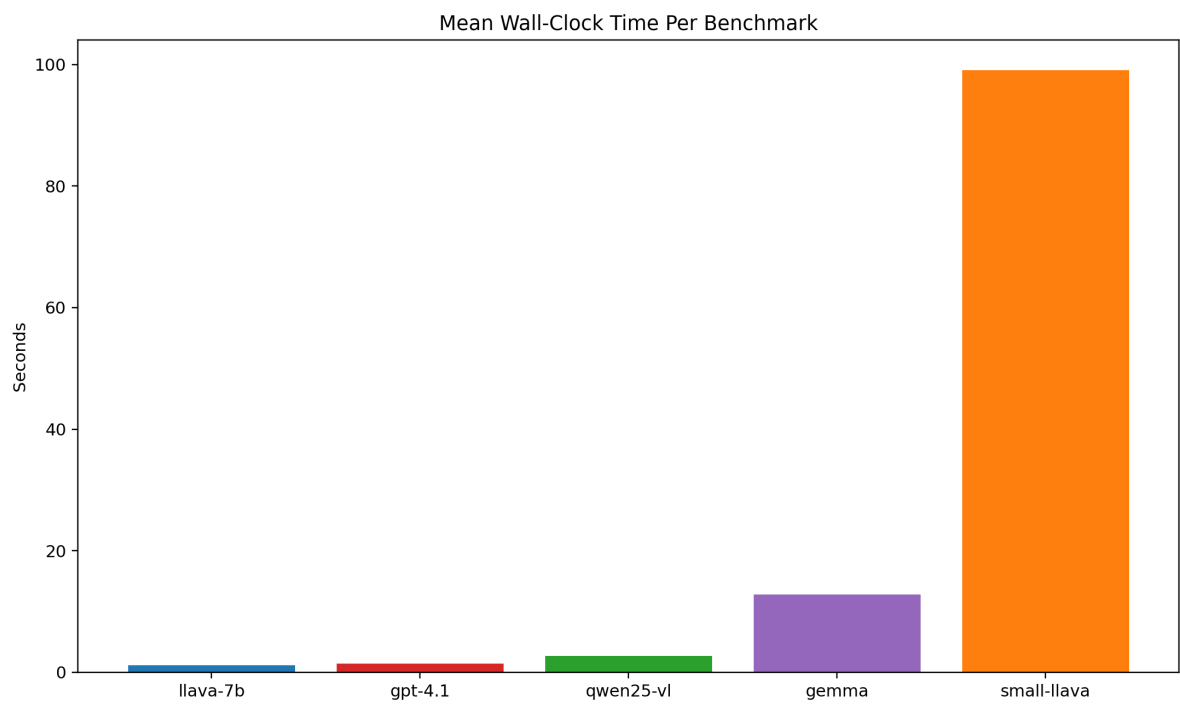


Figure 17: Mean wall-clock time by model.

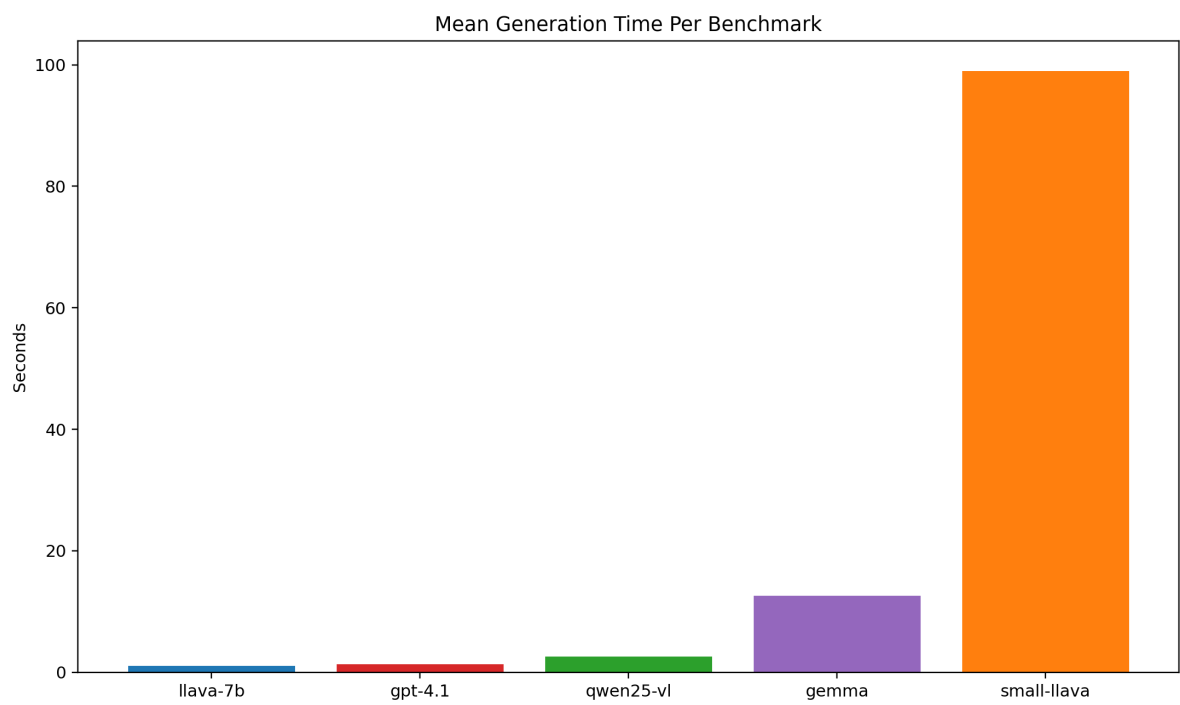


Figure 18: Mean generation time by model.

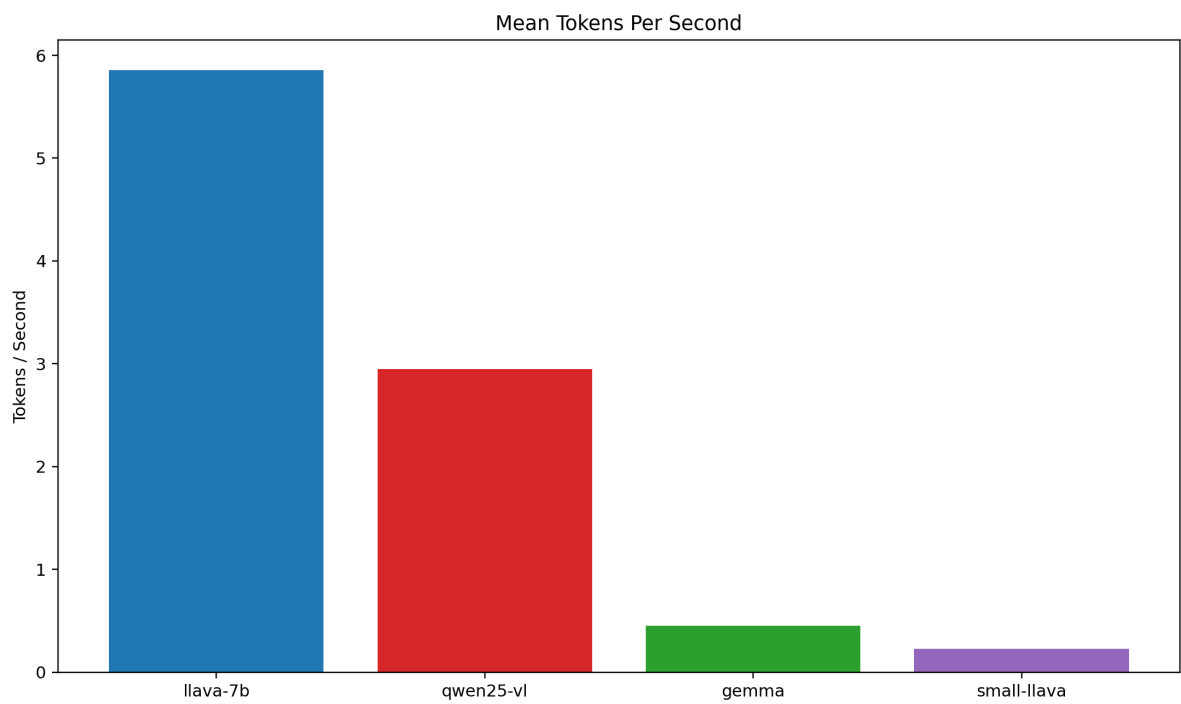


Figure 19: Mean tokens per second by model.

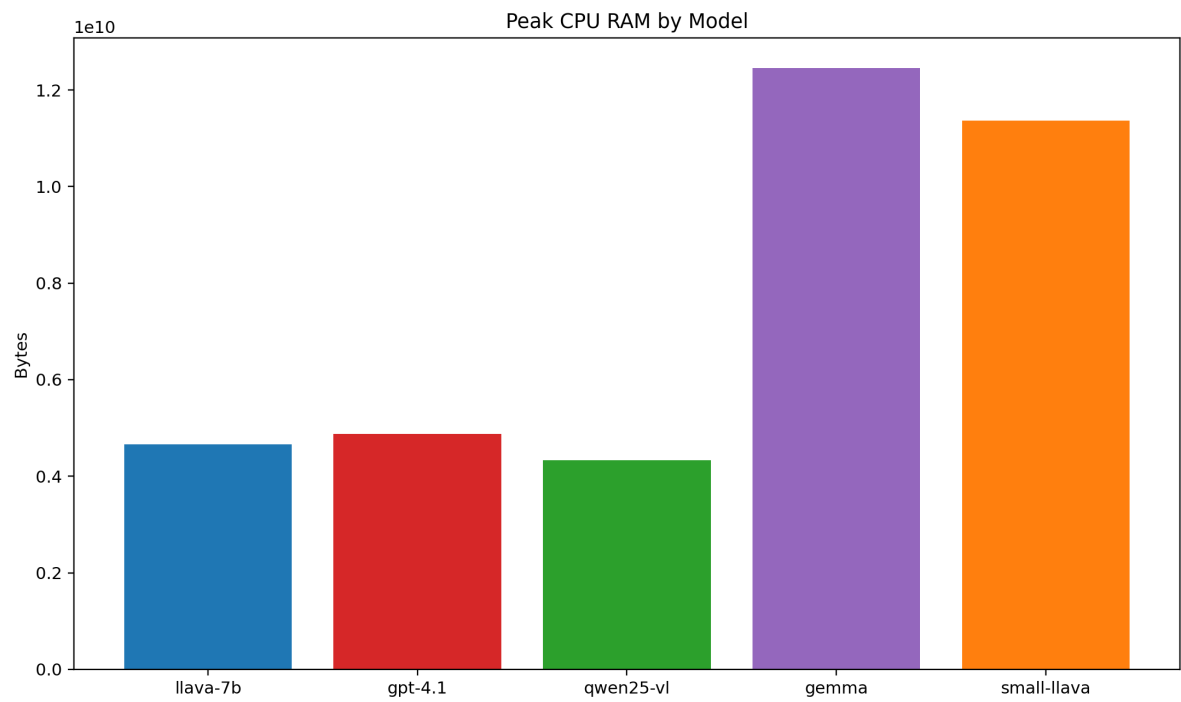


Figure 20: Peak CPU RAM usage by model.

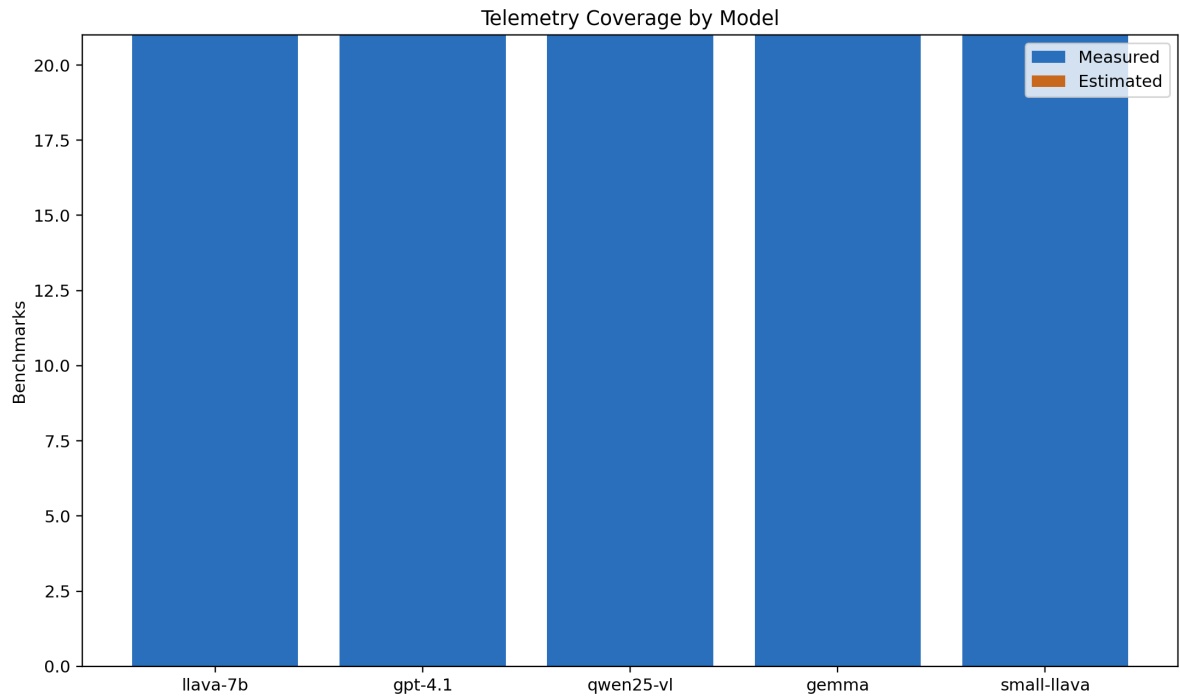


Figure 21: Telemetry coverage across saved benchmark results.

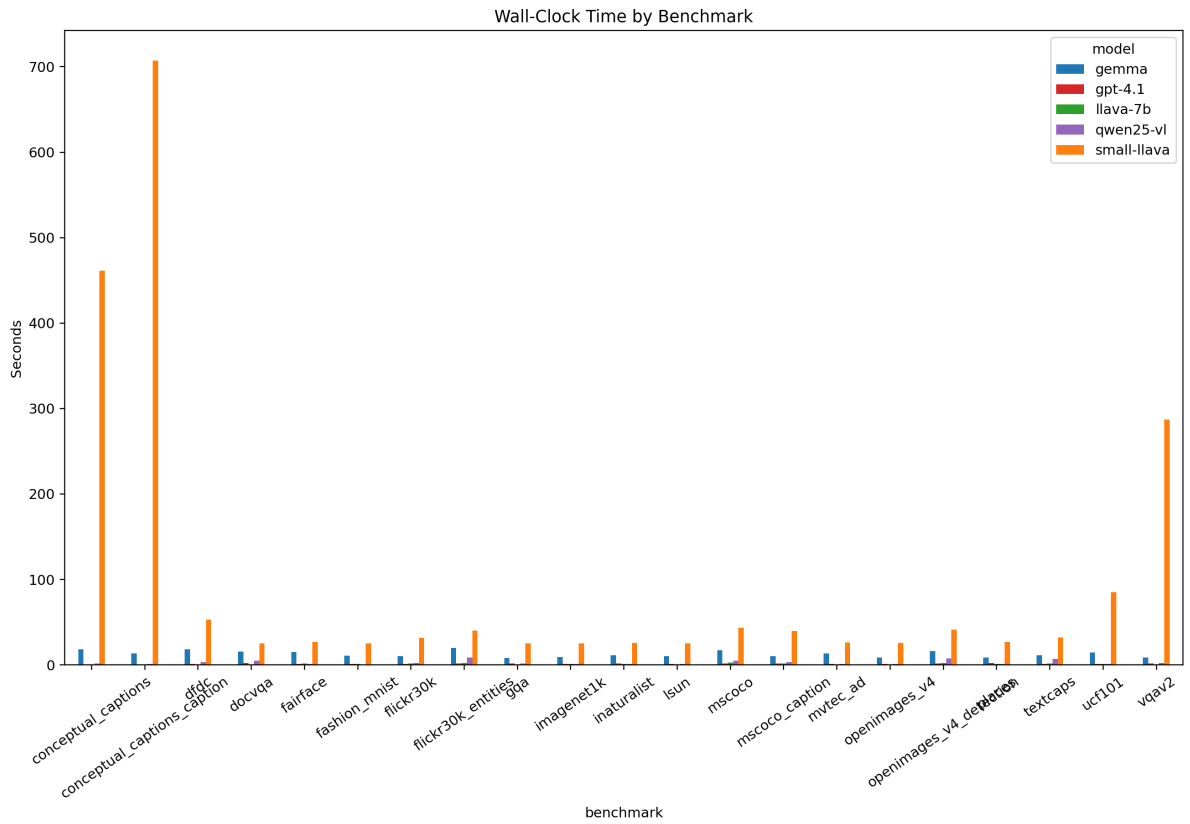


Figure 22: Wall-clock time by benchmark.

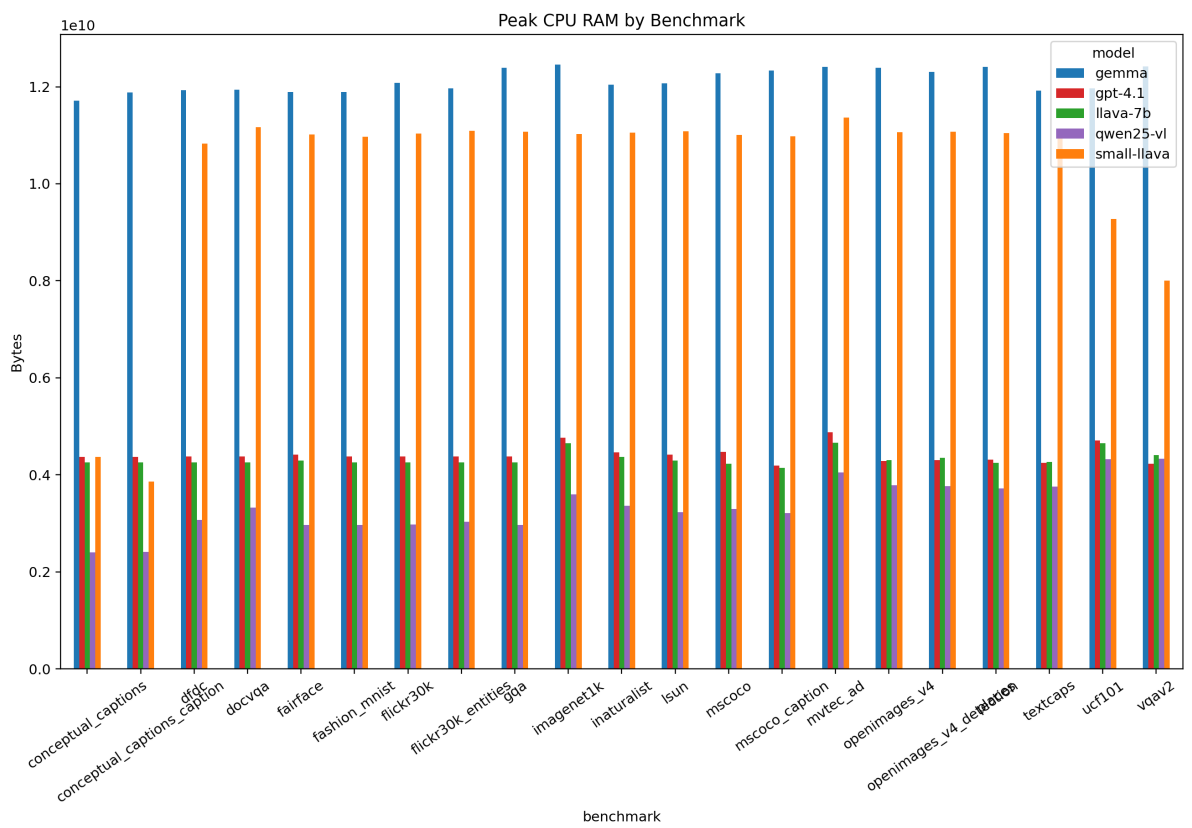


Figure 23: Peak CPU RAM by benchmark.

10 Closing Thought

What I like about this comparison is that it does not produce a cartoonishly simple winner. `gpt-4.1` looks best overall, but it wins by being steadier, not by making the other two irrelevant. `qwen25-vl` is arguably the most impressive pure multimodal competitor here because it wins so often and looks especially natural on captioning. `llava-7b`, meanwhile, still earns its place by showing how much mileage a comparatively straightforward open multimodal stack can get from a good vision encoder, a projector, and a strong language backbone.

That feels like the human version of the result: three good models, three different design philosophies, and performance differences that mostly make sense once you look at how each system turns images into language.